Project Report

Maximiliano Mamone

Project Objective

Exploring classical object detection techniques using LiDAR data from the KITTI 3D Object Detection benchmark and comparing results with the dataset labeled data.

Project Overview

To tackle the problem I started by looking at the data and the format of the LiDAR .bin files, this keeping in mind that labels are in the Camera POV so later for a correct evaluation a transformation will be necessary.

Overall we are working with 7481 training labeled frames from a Velodyne HDL-64E positioned at ~1.7 m from the ground which spins at around 10 Hz producing ~110k-120k points per frame up to 120 m.

Other than that we have calibration for 4 cameras (labels refer to Camera 2 - Colour Left) and the respective Rotation + Rectification + Translation to Camera 0, which is the reference camera in this case.

Pre-processing

First things first, I started a small clean up of the data and the big part in reducing the sheer amount comes also from a useful task which is floor plane removal. To do that I implemented a simple RANSAC algorithm which randomly looks for triplets of points (from a subset with $z < -1.5$) which produce the plane with the most close points associated; close in this case is computed by using the plane definition with each point:

$$s = n \cdot P + d$$

With n being the plane's normal (and normalized) and d the offset. All points within a threshold (0.2 seems to be a reasonable value, also tried 0.1 and 0.15 but 0.2 worked best visually) contribute to the “voting” of the best triplet (the plane with the most inliers). This removes a large share of the points and simplifies not only computing but also the next steps by removing a lot of noise from the data.

Points Clustering

The actual problem of detection comes from how to cluster the points together and to do that I chose to use a Bird Eye View grid of the points, which simplifies the issue by reducing dimensionality and allowing for simple clustering with little compromises. To do that a couple of parameters needed to be set.

Sensor capability indicates a strong degradation after 70 m so I arbitrarily chose 64 m as max distance to evaluate; similarly everything closer than 2 m is also completely ignored given the distance between the sensor and the front of the car.

Other params I arbitrarily set are the number of rows and columns the grid would have (N_R and N_THETA, which ended up respectively at 200 and 1000) and lastly how the points are distributed inside the cells, meaning how the radius of the circles being virtually constructed around the car grow. This is a key point because a linear buildup of the grid would end up with a reducing point density per cell (reason why the logarithmic approach is a common choice and in fact counters this issue pretty well).

Clustering was then done using a king adjacency structure (meaning not only horizontally and vertically adjacent cells were considered the same object but also diagonally) and a morphological closing (3×3) to correct small gaps was also implemented.

Box drawing and classification

For each cluster (≥ 5 points) I fit an oriented rectangle in BEV by projecting all points onto two perpendicular directions for 180 orientations between 0° and 90° (0.5° steps, since a rectangle repeats every 90°) and taking the min/max extents. Instead of the minimum-area rectangle (what cv2.minAreaRect returns), I keep the one whose edges are closest to the points: a car seen from a corner produces an L-shaped cloud, for which the minimum area is ambiguous between the true box and one aligned with the diagonal.

After that the final step was using the z value of the points and the floor previously found to actually draw the 3D box and label the objects found using a simple exclusion approach based on dimensions L×W×H to look for Cars, Cyclists and Pedestrians. Car boxes shorter or narrower than the KITTI mean car (3.9 × 1.6 m) are then extended away from the sensor, since the LiDAR only sees the near side.

Evaluation

Labels are expressed in rectified camera-0 coordinates, so I converted them to the LiDAR frame with the inverse of the calibration chain (R0_rect · Tr_velo_to_cam) and compared boxes there; 3D IoU does not change under a rigid transformation.

The projection matrix P2 (camera 2, the left colour camera the labels refer to) is only used to keep detections inside the camera field of view, since only objects visible in image 2 are labelled.

Detections are matched greedily by decreasing 3D IoU with the official thresholds (0.7 Car, 0.5 Pedestrian/Cyclist) on the Moderate subset (occlusion ≤ 1, truncation ≤ 0.3, 2D height ≥ 25 px). Van and Person_sitting, and objects outside Moderate, are ignored: neither TP nor FN, and a detection on them is not a FP. DontCare regions are not handled, which slightly lowers precision.

Results

Class	TP	FP	FN	Precision	Recall	F1
Car	6237	22473	9466	0.217	0.397	0.281
Pedestrian	321	31022	3248	0.010	0.090	0.018
Cyclist	406	8918	691	0.044	0.370	0.078

Run on 7480 frames, 3D IoU, Moderate difficulty.

AI Use Disclosure

I used an AI assistant as a tutor to improve my understanding of the geometry and methods behind the pipeline (frame transformations, RANSAC, log-polar grids, box fitting, 3D IoU, KITTI evaluation) and to review my code. Data loading, coordinate transformations, ground removal, the BEV grid and clustering, box fitting, 3D IoU, classification rules, label conversion and the evaluation code were written by me, with AI feedback on bugs and then integrated, tested and tuned by me on the data; result_analysis.py (charts and video) was generated almost entirely by AI.